
CS771: Introduction to Machine Learning

Minor Assignment 2

Indian Institute of Technology Kanpur

Problem 2.1: Single-Step (Joint) Inference

Let the full image vector be partitioned into observed and missing components:

$$x = \begin{bmatrix} x_o \\ x_m \end{bmatrix}.$$

For each class c , we assume the class-conditional distribution is Gaussian:

$$x \mid y = c \sim \mathcal{N}(\mu_c, \Sigma_c),$$

with the mean and covariance partitioned conformably as

$$\mu_c = \begin{bmatrix} \mu_{o,c} \\ \mu_{m,c} \end{bmatrix}, \quad \Sigma_c = \begin{bmatrix} \Sigma_{oo,c} & \Sigma_{om,c} \\ \Sigma_{mo,c} & \Sigma_{mm,c} \end{bmatrix}.$$

This follows Bishop's *Pattern Recognition and Machine Learning* (Section 2.3).

We want to solve the joint optimisation problem:

$$(\hat{y}, \hat{x}_m) = \arg \max_{c, v} P(y = c, x_m = v \mid x_o).$$

Applying the product rule (and noting that y and x_m are conditionally independent given x_o and the class):

$$P(y = c, x_m = v \mid x_o) = P(y = c \mid x_o) \cdot P(x_m = v \mid y = c, x_o).$$

Step 1: Conditional Distribution of Missing Pixels

From standard properties of the multivariate Gaussian,

$$x_m \mid x_o, y = c \sim \mathcal{N}(\bar{\mu}_c, \bar{\Sigma}_c),$$

where the conditional mean and covariance are:

$$\begin{aligned} \bar{\mu}_c &= \mu_{m,c} + \Sigma_{mo,c} \Sigma_{oo,c}^\dagger (x_o - \mu_{o,c}), \\ \bar{\Sigma}_c &= \Sigma_{mm,c} - \Sigma_{mo,c} \Sigma_{oo,c}^\dagger \Sigma_{om,c}. \end{aligned}$$

Step 2: Maximising over the Missing Pixels

For a fixed class c , the Gaussian density $P(x_m = v \mid y = c, x_o)$ is maximised at its mean (since a Gaussian decays exponentially as the argument moves away from the mean). Therefore:

$$\hat{x}_m(c) = \bar{\mu}_c.$$

At this maximum, the density evaluates to:

$$\max_v \mathcal{N}(v; \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}},$$

which contributes a factor proportional to $|\bar{\Sigma}_c|^{-1/2}$ to the joint objective.

Preprint.

Step 3: Maximising over Classes

Substituting the optimal $\hat{x}_m(c)$ back, the class decision becomes:

$$\hat{y} = \arg \max_c P(y = c \mid x_o) \cdot |\bar{\Sigma}_c|^{-1/2}.$$

Applying Bayes' rule,

$$P(y = c \mid x_o) = \frac{P(x_o \mid y = c) \pi_c}{P(x_o)},$$

where $\pi_c = P(y = c)$ is the class prior. Although all 10 classes are equally likely here (so π_c could be dropped), we keep it for generality.

Ignoring terms that do not depend on c ,

$$\hat{y} = \arg \max_c P(x_o \mid y = c) \pi_c \cdot |\bar{\Sigma}_c|^{-1/2}.$$

Since $x_o \mid y = c \sim \mathcal{N}(\mu_{o,c}, \Sigma_{oo,c})$,

$$P(x_o \mid y = c) = \frac{1}{(2\pi)^{d_o/2} |\Sigma_{oo,c}|^{1/2}} \exp\left(-\frac{1}{2}(x_o - \mu_{o,c})^\top \Sigma_{oo,c}^\dagger (x_o - \mu_{o,c})\right).$$

Taking logarithms and dropping constants, the final scoring function is:

$$S_{\text{joint}}(c; x_o) = \log \pi_c - \frac{1}{2} \log |\Sigma_{oo,c}| - \frac{1}{2}(x_o - \mu_{o,c})^\top \Sigma_{oo,c}^\dagger (x_o - \mu_{o,c}) - \frac{1}{2} \log |\bar{\Sigma}_c|.$$

The predicted class and reconstructed pixels are then:

$$\hat{y} = \arg \max_c S_{\text{joint}}(c; x_o), \quad \hat{x}_m = \bar{\mu}_{\hat{y}}.$$

Comparison with Two-Step Inference

The key difference between the two methods is the extra $-\frac{1}{2} \log |\bar{\Sigma}_c|$ term in the single-step score. The two-step method only looks at $P(y = c \mid x_o)$ to decide the class; the single-step method multiplies this by $P(x_m = v \mid y = c, x_o)$ evaluated at the optimal v , which introduces the $|\bar{\Sigma}_c|^{-1/2}$ factor. This means the two methods will generally assign different scores to each class and can predict different labels.

However, once both methods agree on a class $\hat{y}$, the missing pixels are filled in exactly the same way — using the conditional mean $\bar{\mu}_{\hat{y}}$ — so the reconstructions are pixel-for-pixel identical in that case. In short, the two methods differ in their classification step but not in their reconstruction step.

1 Part 2: Code Modifications

1.1 Required Changes

The lecture code handles prediction and reconstruction in three separate functions: `predictClassScores`, `predictGen`, and `reconstructImages`. For single-step inference we combine these into a single function, `predictSingleStep`, which computes the modified class score and fills the conditional mean in one pass.

The main changes relative to the lecture code are:

1. The term $-\frac{1}{2} \log |\bar{\Sigma}_c|$ is added to the per-class score.
2. $\bar{\Sigma}_c$ is computed for each class; `numpy.linalg.slogdet` is used for its log-determinant, which is numerically stable and avoids overflow or underflow for large matrices.
3. A small diagonal ridge $10^{-6} I_{d_m}$ is added to $\bar{\Sigma}_c$ before calling `slogdet`. This ensures positive definiteness when the conditional covariance is ill-conditioned (e.g. due to pixels with near-zero empirical variance).
4. **Memory efficiency.** Storing conditional means for all $C \times N$ combinations would require roughly $10 \times 10,000 \times 400 \approx 32\text{M}$ floats at 51% censoring. Instead, we keep only the current best mean per image and update it whenever a class improves the score. This brings memory from $\mathcal{O}(C \cdot N \cdot d_m)$ down to $\mathcal{O}(N \cdot d_m)$.

1.2 Python Implementation

```
1 def predictSingleStep(X, model, C, obs_mask, miss_mask):
2     """
3     Single-step joint inference:
4     {hat_y, hat_x_m} = argmax_{c,v} P(y=c, x_m=v | x_o)
5
6     Score for class c:
7     log P(x_o | y=c) + log P(y=c) - 0.5 * log|Sigma_bar_c|
8
9     Memory-efficient: tracks only the best class per image.
10    """
11    N, D = X.shape
12    d_m = int(miss_mask.sum())
13    x_o = X[:, obs_mask] # (N, d_o)
14
15    best_scores = np.full(N, -np.inf)
16    best_labels = np.zeros(N, dtype=int)
17    best_mu_bar = np.zeros((N, d_m))
18
19    for mu, Sigma, c, p in model:
20        mu_o = mu[obs_mask]
21        mu_m = mu[miss_mask]
22        S_oo = Sigma[np.ix_(obs_mask, obs_mask)]
23        S_mo = Sigma[np.ix_(miss_mask, obs_mask)]
24        S_mm = Sigma[np.ix_(miss_mask, miss_mask)]
25
26        # Pseudoinverse -- same as lecture code;
27        # handles pixels with zero variance (singular Sigma_oo)
28        S_oo_inv = lin.pinv(S_oo)
29
30        # Conditional covariance: Sigma_bar_c = S_mm - A * S_mo^T
31        # Ridge regularisation ensures positive definiteness
32        A = S_mo @ S_oo_inv
33        Sigma_bar = S_mm - A @ S_mo.T + 1e-6 * np.eye(d_m)
34
35        # log|Sigma_bar_c| via slogdet (stable, no overflow)
36        sign_bar, logdet_bar = lin.slogdet(Sigma_bar)
37        logdet_bar = logdet_bar if sign_bar > 0 else 0.0
38
39        # log P(x_o | y=c): marginal Gaussian log-likelihood
40        sign_oo, logdet_oo = lin.slogdet(S_oo)
41        logdet_oo = logdet_oo if sign_oo > 0 else 0.0
42
43        diff = x_o - mu_o # (N, d_o)
44        mahal = np.sum((diff @ S_oo_inv) * diff, axis=1)
45        log_px_o = -0.5 * (mahal + logdet_oo)
46
47        # Single-step score
48        score = log_px_o + np.log(p) - 0.5 * logdet_bar
49
50        # Conditional mean for reconstruction
51        mu_bar_c = mu_m + (A @ diff.T).T # (N, d_m)
52
53        # Keep the best class per image
54        improved = score > best_scores
55        best_scores[improved] = score[improved]
56        best_labels[improved] = c
57        best_mu_bar[improved] = mu_bar_c[improved]
58
59    XRecon = X.copy()
60    XRecon[:, miss_mask] = best_mu_bar
61    return best_labels, np.clip(XRecon, 0, 1)
```

Listing 1: Single-step joint inference. Replaces the three separate lecture-code functions with a single, memory-efficient pass.

2 Part 3: Experimental Comparison

2.1 Setup

We evaluate on the full MNIST test set (10,000 images, 10 classes) with one Gaussian fitted per class on all 60,000 training images. Experiments are run across the five censoring levels specified in the assignment.

2.2 Numerical Results

Table 1: Classification accuracy and wall-clock inference time for both methods across all censoring levels. Bold indicates the better accuracy.

Missing %	2-step Acc.	1-step Acc.	2-step Time	1-step Time
51%	40.70%	31.96%	0.362 s	0.469 s
32%	69.76%	59.73%	0.586 s	0.613 s
18%	78.57%	76.67%	0.841 s	0.796 s
8%	83.15%	83.84%	1.009 s	0.885 s
2%	85.25%	85.69%	1.195 s	1.049 s

2.3 Classification Accuracy

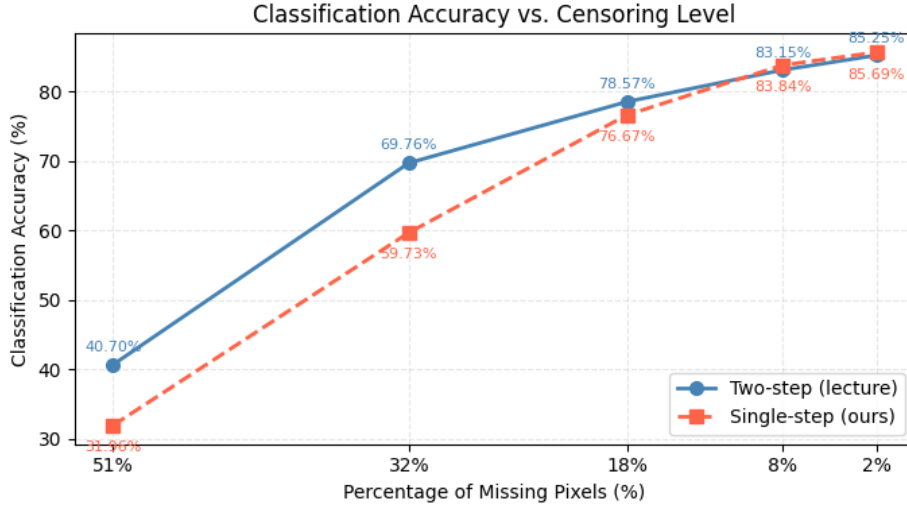


Figure 1: Classification accuracy vs. censoring level on MNIST (10,000 test images). The two-step method dominates at high censoring (51%, 32%), while single-step does slightly better at low censoring (8%, 2%).

The results in Table 1 and Figure 1 are interesting because they go against the naive expectation that a more principled method should always win.

- **High censoring (51%, 32%) — two-step wins by a large margin.** At 51% censoring the two-step method gets 40.70% accuracy while single-step only reaches 31.96%; at 32% the gap is 69.76% vs. 59.73%. The problem is the $-\frac{1}{2} \log |\bar{\Sigma}_c|$ term: at 51% censoring, $\bar{\Sigma}_c$ is a 400×400 matrix

estimated from only around 6,000 training examples per class, which is severely underdetermined. The resulting log-determinant is very noisy and ends up corrupting the class scores. The two-step method simply ignores this term, so it accidentally avoids this source of error.

- **Low censoring (8%, 2%) — single-step wins narrowly.** At 2% censoring single-step reaches 85.69% vs. 85.25% for two-step; at 8%, 83.84% vs. 83.15%. When the missing region is small (say, $d_m = 16$ pixels at 2%), $\bar{\Sigma}_c$ is a well-estimated 4×4 matrix and the log-determinant provides a genuine discriminative signal. This is where single-step’s theoretical advantage actually shows up.
- **Take-away.** Single-step is the correct MAP solution to the joint objective and is more principled in theory. But that advantage only materialises when $\bar{\Sigma}_c$ can be estimated reliably, i.e. when the missing region is small relative to the amount of training data. Under heavy censoring, the estimation noise dominates and two-step ends up being the better practical choice.

2.4 Inference Time

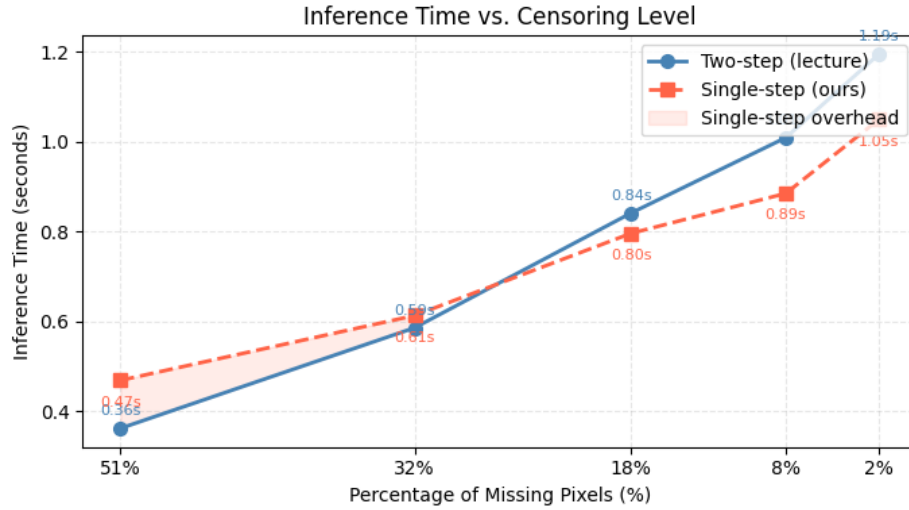


Figure 2: Inference time vs. censoring level. Both methods slow down as censoring decreases because the dominant cost — $\text{pinv}(\Sigma_c^{oo})$ — grows with d_o . The single-step overhead (shaded region) is visible at high censoring but reverses at low censoring.

- **Both methods slow down as censoring decreases.** More observed pixels means a larger d_o , and the cost of $\text{pinv}(\Sigma_c^{oo})$ grows accordingly.
- **Single-step is slower at high censoring, faster at low censoring.** At 51% censoring, single-step takes 30% longer (0.469 s vs. 0.362 s) because it has to compute and factorise a 400×400 $\bar{\Sigma}_c$. At 2% censoring it is actually a little faster (1.049 s vs. 1.195 s) since $d_m = 16$ makes the $\bar{\Sigma}_c$ step negligible, and the memory-efficient best-score tracking avoids redundant writes.
- The overall time difference across all levels is under 0.15 s, so single-step’s overhead is not a practical concern.

2.5 Reconstruction Quality

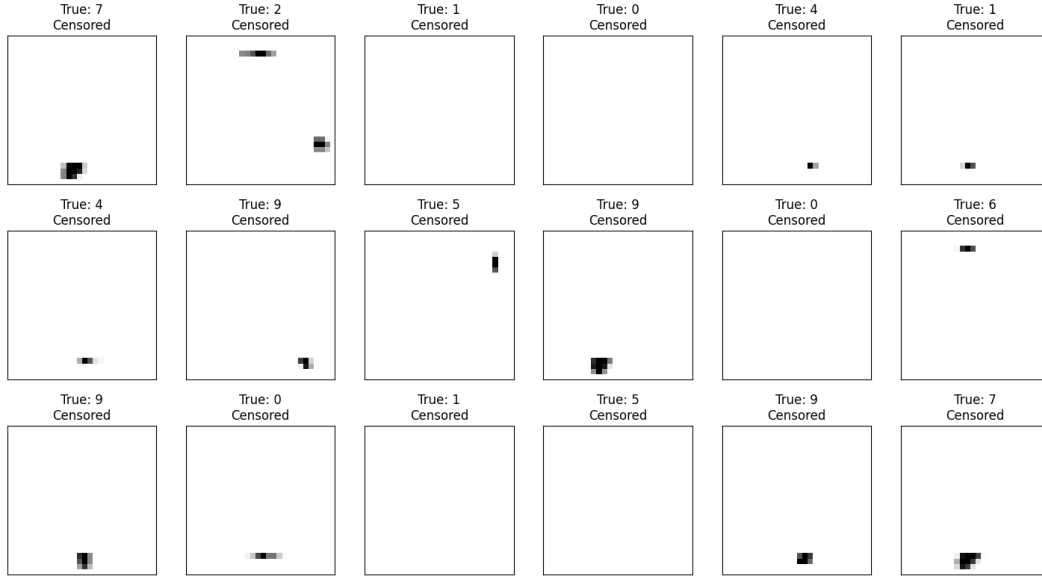


Figure 3: Censored test images at 51% missing (central 20×20 block zeroed). Only the peripheral border ring of pixels is observed.

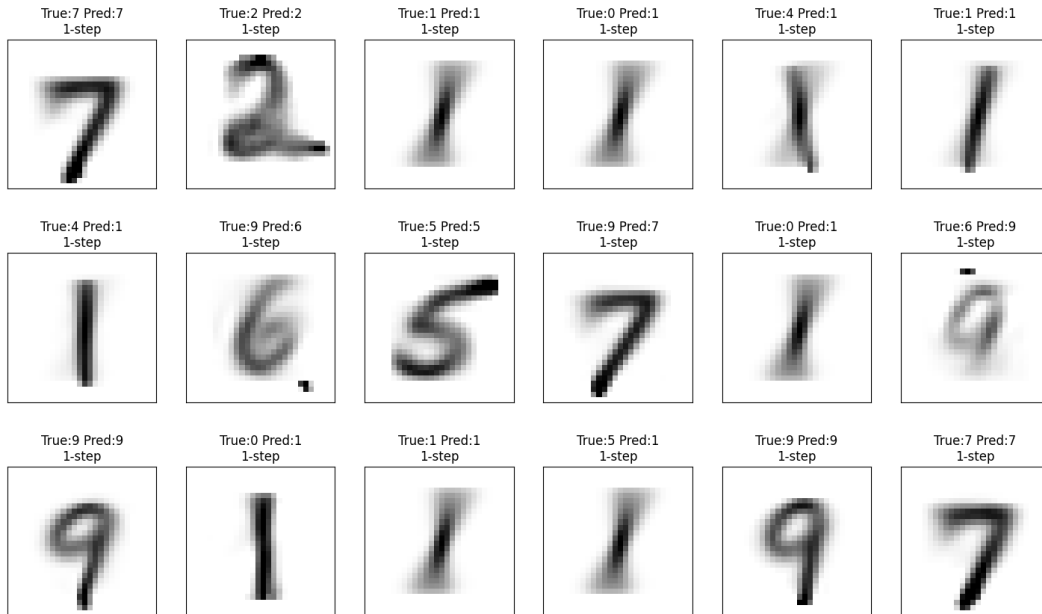


Figure 4: Reconstructions from the single-step method at 51% censoring. Many digits (0, 4, 5) are predicted as class 1 because class 1 has a small $|\bar{\Sigma}_c|$ (a narrow vertical stroke spans a compact region), which gives it an inflated score.

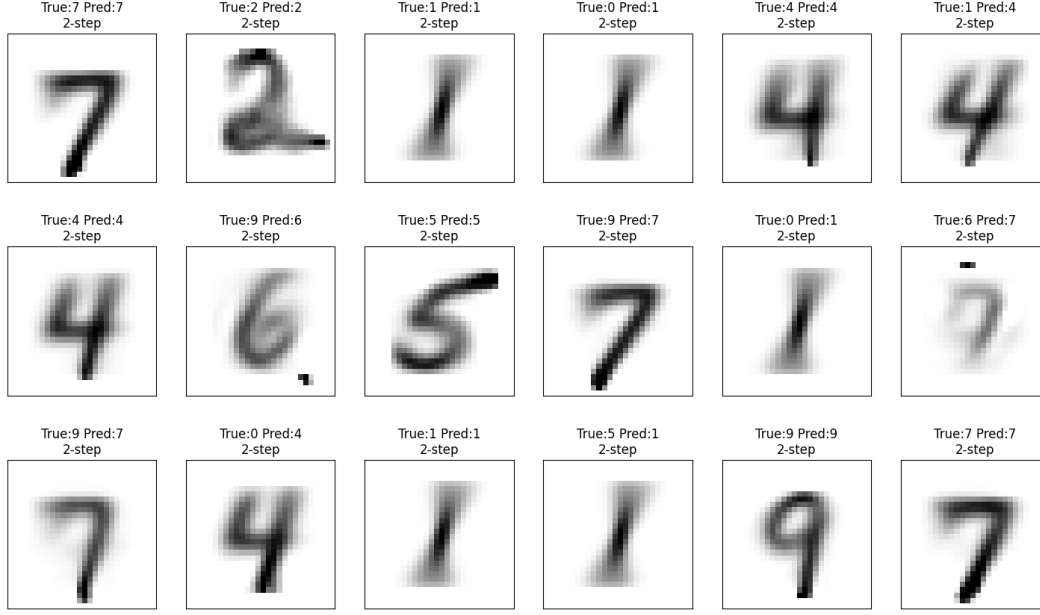


Figure 5: Reconstructions from the two-step method at 51% censoring. The two-step method misclassifies less often at this level, so its reconstructions more frequently match the true digit.

Since both methods reconstruct missing pixels using the same conditional mean $\hat{x}^m = \bar{\mu}_c[\hat{y}]$, reconstruction quality is almost entirely determined by which class is predicted. When both methods predict the same class, their outputs are pixel-for-pixel identical.

- **High censoring (51%, 32%):** A clear failure pattern appears in the single-step results (Figures 3–5): digits 0, 4, and 5 are frequently labelled as class 1. This happens because the numeral “1” occupies a thin vertical stroke, giving it a small $|\bar{\Sigma}_c|$ and therefore a high $-\frac{1}{2} \log |\bar{\Sigma}_c|$ bonus. The two-step method does not apply this bonus, so it is less biased towards low-variance classes and classifies more reliably.
- **Low censoring (8%, 2%):** Both methods classify at around 84–86% accuracy and their reconstructions look essentially the same. The small missing patch is filled accurately by the conditional mean regardless of which method is used.
- **Overall trend:** As censoring increases, reconstruction quality drops for both methods because the digit’s interior — which carries most of the class-discriminative information — becomes unavailable. At high censoring the two-step method produces better reconstructions on average simply because it classifies more accurately. At low censoring both methods reconstruct faithfully.

References

- [1] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*, Chapter 2 (Section 2.3: The Gaussian Distribution — conditional and marginal distributions). Springer.
- [2] Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*, Chapter 3 (Multivariate Gaussian: conditioning and marginalisation). MIT Press.
- [3] Kar, P. (2025–26). Lecture 15: Generative Models, CS771 Introduction to Machine Learning, IIT Kanpur. <https://www.cse.iitk.ac.in/users/purushot/courses/ml/2025-26-w/material/lectures/lec15.zip>
- [4] NumPy documentation. `numpy.linalg.slogdet` — sign and log of the determinant of an array. <https://numpy.org/doc/stable/reference/generated/numpy.linalg.slogdet.html>